

Számítási és tárolási felhők alapjai (11. hét)

Big Data alapok

Szeberényi Imre, Ludmány Balázs

BME IIT

<szebi@iit.bme.hu>



MŰEGYETEM 1782

Miért a felhőben?



- Gyors kipróbálási lehetőség
- Skálázhatóság
- Terheléselosztás
- Adatok keletkezési helye egyre többször a felhő
- Egyszerűbb klasztermanagement

Üzleti szereplők

- Google Compute Engine (Hadoop)
- Amazon Big data analytics (Hadoop)
- RackSpace (Hadoop)
- Oracle Cloud Big data
- IBM Big data in the cloud (Cloudbant, dashDB, VoltDB)
- HP Big data (Cassandra, Hadoop)
- MS (Hadoop)

Üzleti szereplők (2025)

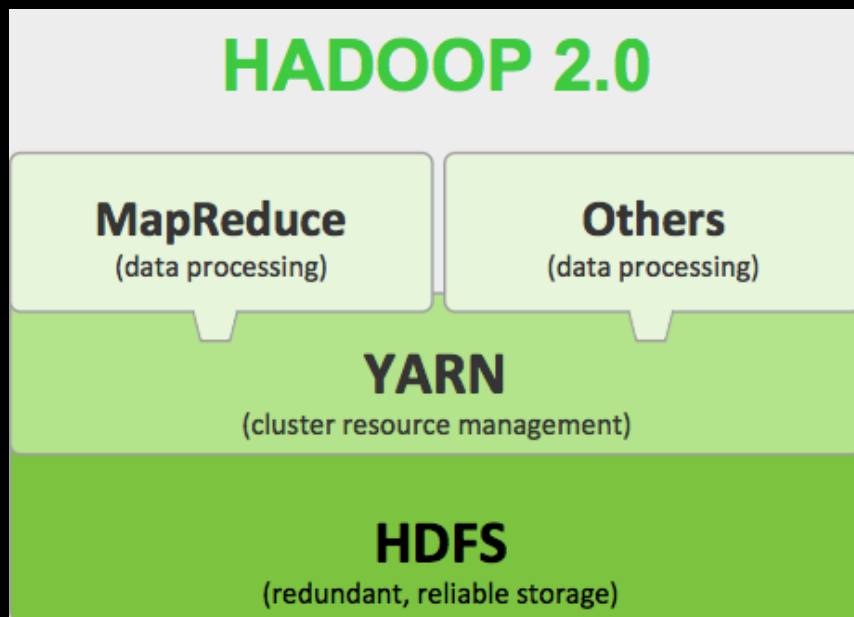
- AWS – EMR, EMR Serverless, Glue
- Google Cloud – Dataproc, BigQuery
- Microsoft Azure – HDInsight, Synapse Analytics
- Databricks – Lakehouse platform
- Snowflake – Cloud Data Platform

Apache Hadoop

- Keretrendszer nagyméretű adatok feldolgozásához
 - klaszter bázisú
 - elosztott
 - hibatűrő (sw. rétegben)
 - skálázható
- Számos sw. komponens együtt
 - Hadoop Common
 - HDFS – elosztott fájlrendszer
 - YARN – elosztott job ütemező
 - MapReduce – párhuzamos feldolgozó keretrendszer
 - Spark/Flink – mapReduce helyett

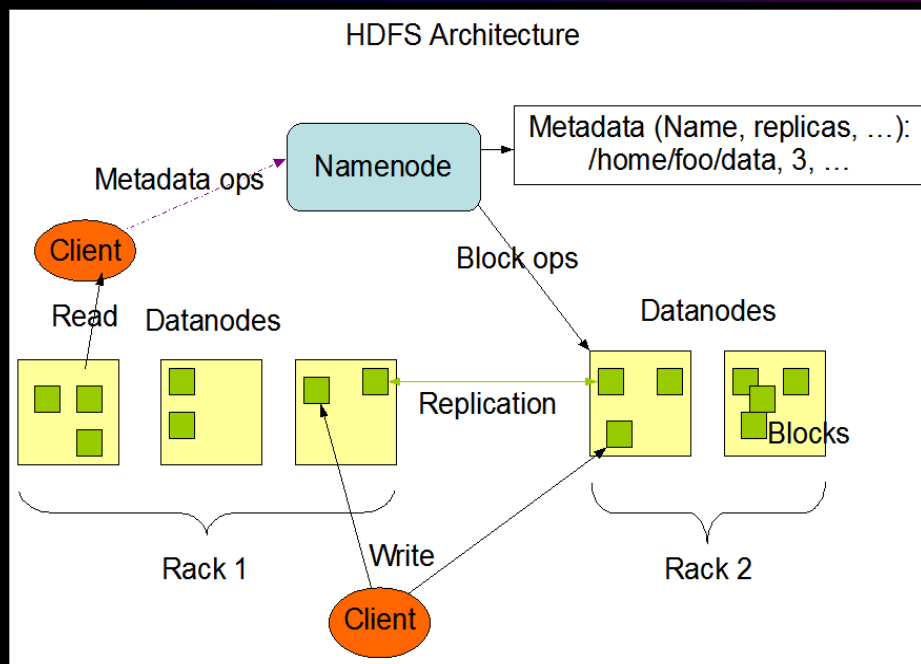


Hadoop fő komponensei



- Hadoop common a modulok kezeléséért, a közös funkciók megvalósításáért felel.
- HDFS: Elosztott, hibatűrő, nagy fájlok tárolására alkalmas FS.
- YARN – erőforrás menedzser és job ütemező. HDFS-sel együttműködve a job-ot az adathoz közel futtatja.
- MapReduce – Keretrendszer map-reduce programok készítésére és futtatására.
- Aktuális verzió 2025-ben 3.4.2

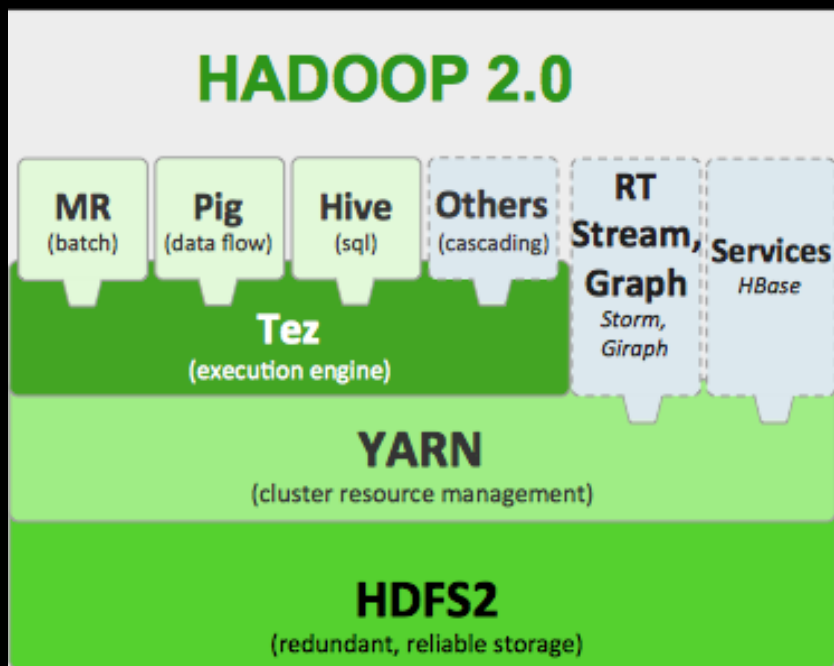
HDFS és komponensei



- Namenode – metadata tárolásáért felelős.
- Datanode – adatblokkok tárolásáért felelős.
- Adatblokkok többszörös replikával tárolhatók.
- Secondary NameNode vs. Standby (HA) NameNode

- Write once, Read-many felhasználásra, és batch feldolgozásra optimalizált megvalósítás.
- Standby namenode + JournalNode segítségével tovább növelhető a megbízhatóság

Kapcsolódó Apache projektek

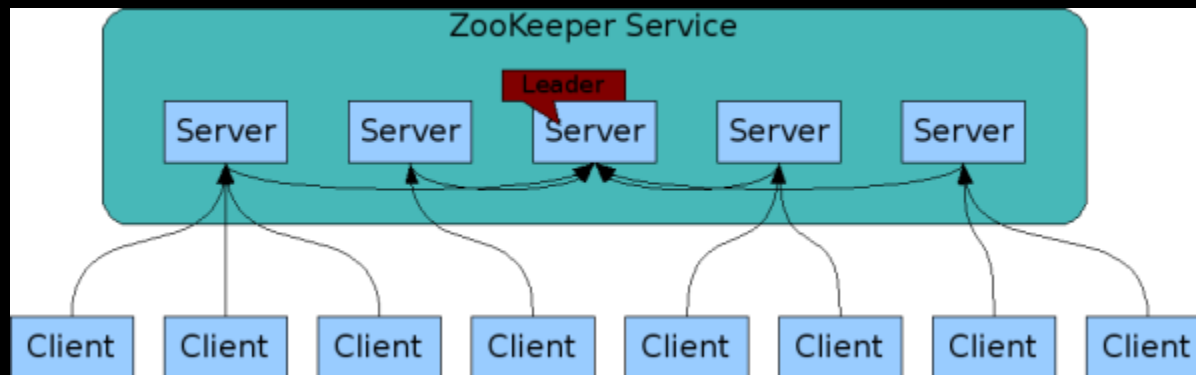


- Pig – MR script nyelv
- Hive – Petabájt méretű adattárház.
- Tez – DAG bázisú MR processz-menedzsment.
- Hbase – NoSQL adatbázis motor.

- Avro – adatsorosító keretrendszer
- Cassandra – elosztott adatbázisszerver
- Chukwa – elosztott log gyűjtő és elemző
- Mahout – elosztott tanuló rendszer

Kapcsolódó Apache projektek /2

- Spark – SQL + analytics
- Flink – Valós idejű streaming
- ZooKeeper – keretrendszer elosztott programok készítéséhez (konfiguráció és telepítés menedzsment, szinkronizációs eszköz)
 - Hierarchikus névtérben tárolt adatok



Projektek kategorizálva

Adattárolás és fájlrendszer

- HDFS – elosztott, hibátűrő fájlrendszer
- Ozone – S3-szerű skálázható objektumtároló

Adatfeldolgozás

- Apache Spark – gyors batch + SQL + gépi tanulás
- Apache Flink – valós idejű, stateful streaming és batch
- Apache Tez – DAG-alapú futtatómotor (Hive működési alapja)

NoSQL és koordináció

- HBase – elosztott, oszlopalapú NoSQL adatbázis
- ZooKeeper – elosztott koordináció, lokkolás, konfiguráció (kis fájlok)

Projektek kategorizálva #2

Adatformátumok és táblarendszerek

- Avro – adatsorosítás
- Parquet, ORC – oszlopos, tömörített adattárolás
- Iceberg / Hudi / Delta Lake – modern táblakezelés nagy adathalmazokhoz

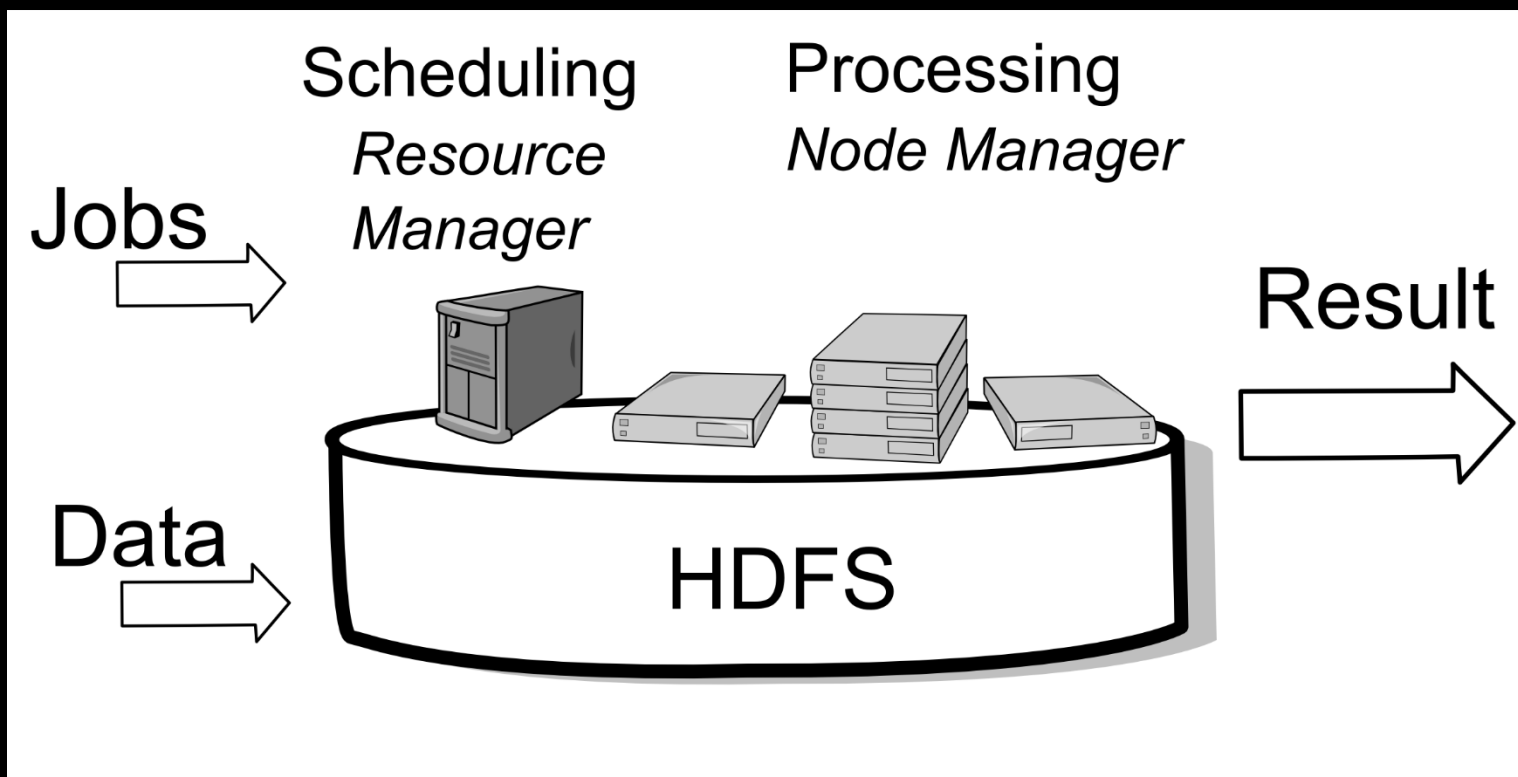
SQL + metaadat (schema-on-demand)

- Hive – SQL lekérdező réteg nagy fájlokra (HDFS/S3)
- Impala – nagyon gyors SQL lekérdező motor
- Presto / Trino – elosztott SQL motor több adatforrásra

Régiiek / csökkenő jelentőségűek

- Pig – MR-generáló scriptnyelv (gyakorlatilag deprecated)
- MapReduce – klasszikus batch keretrendszer (oktatási cél)
- Mahout – korábbi elosztott ML könyvtár (Spark/Flink váltotta ki)

MR adatfeldolgozás vázlat



MapReduce #1

Keretrendszer nagy adathalmaz párhuzamos feldolgozására

- Map

- Adatok valamilyen szempont szerinti (kulcs) darabolása (párhuzamosan).

- Reduce

- Az azonos kulcsú adatok összesítése (számlálás, összefűzés, stb.)

A két fázis között kulcs szerinti rendezés történik, így a Reduce az azonos kulcsú adatokat kapja meg.

MapReduce #2

Java keretrendszer 23 osztályt és 7 interfészt definiál. Ezek közül a legfontosabbak:

- **Job:** A feladatot leíró objektum amelyet a user konfigurál.
- **Mapper:** A map műveletet megvalósító objektum.
- **Reducer:** Három fő működési fázisa van: a **Shuffle** fázisban HTTP-vel begyűjti a mapperekről az inputot, a **Sort** fázisban ezeket mergeli, majd a **Reduce** fázisban feldolgozza.
- **InputFormat:** Az input formátumot leíró objektum.
- **InputSplit:** Az mapperek által kapott input halmaz.

MR példa: szószámláló #1

Map: Tokenizál, → kulcs → <kulcs, 1>

```
private final static IntWritable one = new IntWritable(1);
public void map(Object key, Text value, Context context) ... {
    StringTokenizer itr = new StringTokenizer(value.toString());
    while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
    }
}
```

MR példa: szószámláló #2

Reduce: kulcsokhoz tartozó értékek összeadása

```
private IntWritable result = new IntWritable();  
public void reduce(Text key, Iterable<IntWritable> values,  
                   Context context) ... {  
    int sum = 0;  
    for (IntWritable val : values) {  
        sum += val.get();  
    }  
    result.set(sum);  
    context.write(key, result);  
}
```


Pig

- Nagy adathalmazok elemzése
- Script nyelv (Pig Latin)
- Map-Reduce programot állít elő
- A végrehajtás párhuzamosítható
- Működési módok:
 - Local
 - Tez
 - Mapreduce



pig.apache.org

Pig koncepció

- Load
- Foreach _____ Generate _____
- Filter ____ BY ____
- Group ____ BY ____
- Store



Pig Latin (nem a nyelvi játék)

- Relations: bag (outer bag)
 - Táblához hasonló
- Bag: unordered collection of tuples
- Tuple: ordered set of fields
 - Sorokhoz hasonló, de eltérő számú és típusú mezők is lehetnek.
- Field: piece of data

$(1, \{ (1,2,3) \})$

$(4, \{ (4,2,1), (4,3,3,) \})$

$(8, \{ (8,3,4) \})$

Pig péda

Input: eladási adatok

alma	1	kg	150	2014-12-05:12:00
tej	2	l	180	2014-11-15:22:10
tojás	3	db	45	2013-12-05:12:50
alma	5	kg	120	2014-10-25:02:20
korte	3	kg	280	2014-12-05:18:11

Feladat:

- Összes bevétel
- Átlag, napi átlag
- Csak az alma

Adatok betöltése

```
A = LOAD 'file:data.txt' AS (name:chararray,  
quantity:int, unit:chararray, price:double,  
date:chararray);
```

```
DUMP A;
```

```
(alma,1,kg,150.0,2014-12-05:12:00)
```

```
(tej,2,l,180.0,2014-11-15:22:10)
```

```
(tojás,3,db,45.0,2013-12-05:12:50)
```

```
(alma,5,kg,120.0,2014-10-25:02:20)
```

```
(korte,3,kg,280.0,2014-12-05:18:11)
```

Dátum átalakítása

B = FOREACH A GENERATE

name,quantity,unit,price,

ToDate(date, 'yyyy-MM-dd:HH:mm') AS date;

DUMP B;

(alma,1,kg,150.0,2014-12-05T12:00:00.000+01:00)

(tej,2,l,180.0,2014-11-15T22:10:00.000+01:00)

(tojás,3,db,45.0,2013-12-05T12:50:00.000+01:00)

(alma,5,kg,120.0,2014-10-25T02:20:00.000+02:00)

(korte,3,kg,280.0,2014-12-05T18:11:00.000+01:00)

Eladási nap és összeg

```
C = FOREACH B GENERATE name,  
price*quantity AS total,  
ToUnixTime(date)/86400 AS day;  
DUMP C;
```

(alma,150.0,16409)

(tej,360.0,16389)

(tojás,135.0,16044)

(alma,600.0,16368)

(korte,840.0,16409)

Csoportosítás

D = GROUP C ALL;

E = GROUP C BY day;

DUMP D;

(all,{(korte,840.0,16409),(alma,600.0,16368),(tojas,135.0,16044),(tej,360.0,16389),(alma,150.0,16409)})

DUMP E;

(16044,{(tojas,135.0,16044)})

(16368,{(alma,600.0,16368)})

(16389,{(tej,360.0,16389)})

(16409,{(korte,840.0,16409),(alma,150.0,16409)})

Összegzés, átlag

```
T = FOREACH D GENERATE SUM(C.total) AS  
tot, AVG(C.total) AS avg;
```

```
DUMP T;
```

```
(2085.0,417.0)
```

```
TT = FOREACH E GENERATE group,  
SUM(C.total) AS tot, AVG(C.total) AS avg;
```

```
DUMP TT;
```

```
(16044,135.0,135.0)
```

```
(16368,600.0,600.0)
```

```
(16389,360.0,360.0)
```

```
(16409,990.0,495.0)
```

Használt relációk (illustrate E)

```
-----
| A | name:chararray | quantity:int | unit:chararray | price:double   | date:chararray   |
-----
|   | korte          | 3           | kg             | 280.0          | 2014-12-05:18:11 |
|   | alma            | 1           | kg             | 150.0          | 2014-12-05:12:00 |
-----
```

```
-----
| B | name:chararray | q:int | unit:chararray | price:double | date:datetime           |
-----
|   | korte          | 3     | kg             | 280.0        | 2014-12-05T18:11:00.000+01:00 |
|   | alma            | 1     | kg             | 150.0        | 2014-12-05T12:00:00.000+01:00 |
-----
```

```
-----
| C | name:chararray | total:double | day:long       |
-----
|   | korte          | 840.0        | 16409          |
|   | alma            | 150.0        | 16409          |
-----
```

```
-----
| E | group:long | C:bag{tuple(name:chararray,total:double,day:long)} |
-----
|   | 16409       | {(korte, 840.0, 16409), (alma, 150.0, 16409)} |
-----
```

Hive



- Hadoop alapú adattárház
- SQL-szerű lekérdező nyelv
- Map-Reduce programot állít elő
- Adatok HDFS-en
- Meta adatok:
 - saját adatbázisban,
 - RDBMS-ben (mysql, postgres)
 - HBase

hive.apache.org

Hive alapfogalmak



- Adatbázisok
- Táblák
- Partíciók
 - Táblák partíciókra oszthatók. Alapvetően tárolási fogalom, de hatékonyan felhasználható a lekérdezésekben.
- Vödrök (buckets/clusters)
 - Minden partíció vödrökre bontható

Vördök és partíciók

```
CREATE TABLE user_info(user_id BIGINT,  
firstname STRING, lastname STRING)  
COMMENT 'A bucketed copy of user_info'  
PARTITIONED BY(ds STRING)  
CLUSTERED BY(user_id) INTO 256 BUCKETS;
```

```
FROM user_id  
INSERT OVERWRITE TABLE user_info  
PARTITION (ds='2009-02-25')  
SELECT userid, firstname,  
lastname WHERE ds='2009-02-25';
```

Hive fájlformátumok

- Text
- Avro – serializálható fájlformátum
- ORC – Optimized Row Columnar
- Parquet – Tömörített oszlopos
- Compressed Data – gzip, bzip
- LZO tömörített

Hive példafeladat

Input:

- Apach logfile-ok a HDFS-en:

199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245

burger.letters.com - - [01/Jul/1995:00:00:12 -0400] "GET /NASA-logosmall.gif HTTP/1.0" 304 0

Feladat:

- Kérésszám statisztika
- Statisztika a visszatérő látogatókról
- Hosztonkénti statisztika az utolsó két letöltésről

input tábla létrehozása

```
USE DB_NEPTUN;  
CREATE EXTERNAL TABLE access (  
    host STRING, identity STRING, user STRING,  
    time STRING, request STRING, status STRING,  
    size STRING, referer STRING) ROW FORMAT SERDE  
'org.apache.hadoop.hive.serde2.RegexSerDe' WITH  
SERDEPROPERTIES (  
    "input.regex" = "([^ ]*) ([^ ]*) ([^ ]*) (-|\\[[^\\]]*\\]) ([^  
\\"]*|\"[^\"]*\\") (-|[0-9]*) (-|[0-9]*)(.*)",  
    "output.format.string" = "%1$$ %2$$ %3$$ %4$$ %5$$  
%6$$ %7$$ %8$$")  
STORED AS TEXTFILE LOCATION '/szebi/NASA';
```


access2 létrehozása

```
CREATE EXTERNAL TABLE access2 (  
  host STRING, -- name or ip address of the host  
  time BIGINT, -- unix time, utc.  
  method STRING, -- "get", etc.  
  path STRING, -- "/logo.png", etc.  
  protocol STRING, -- "http/1.1", etc.  
  status SMALLINT, -- 200, 404, etc.  
  size BIGINT) -- response size, in bytes.  
STORED AS SEQUENCEFILE;
```

Adatok betöltése access2-be

```
INSERT OVERWRITE TABLE access2
SELECT
  host,
  UNIX_TIMESTAMP(time, "[dd/MMM/yyyy:HH:mm:ss Z]"),
  REGEXP_EXTRACT(request, "([^ ]*) ([^ ]*) ([^\\"]*)", 1) AS method,
  REGEXP_EXTRACT(request, "([^ ]*) ([^ ]*) ([^\\"]*)", 2) AS path,
  REGEXP_EXTRACT(request, "([^ ]*) ([^ ]*) ([^\\"]*)", 3) AS protocol,
  CAST(status AS SMALLINT) AS status,
  CAST(size AS BIGINT) AS size
FROM access WHERE UNIX_TIMESTAMP(time,
                                "[dd/MMM/yyyy:HH:mm:ss Z]") IS NOT NULL;
```

Kérésszámok statisztikája



```
SELECT INNERTABLE.dcount AS downloads,  
COUNT(INNERTABLE.host) AS count FROM (  
    SELECT COUNT(*) AS dcount, host FROM access2  
    WHERE PATH LIKE '%.htm%' GROUP BY host  
) INNERTABLE GROUP BY INNERTABLE.dcount;
```

Visszatérő látogatók

```
SELECT host, count FROM (  
  SELECT host, COUNT(host) AS count, MIN(time) AS minimum,  
    MAX(time) AS maximum, MAX(time) - MIN(time) AS delta  
  FROM access2 WHERE PATH LIKE '%.htm%' GROUP BY  
    host  
) T WHERE T.delta > 21600; -- 6 óránál több idő telt el
```

Utolsó 2 lekérdezés ideje

```
A = LOAD 'access2' USING
org.apache.hive.hcatalog.pig.HCatLoader();
GROUPED = GROUP A BY host;
TOP3 = FOREACH GROUPED {
    SORTED = ORDER A.time BY time DESC;
    TOP = LIMIT SORTED 2;
    X = FOREACH TOP GENERATE ToDate($0*1000);
    GENERATE group, X;
};
B = LIMIT TOP3 10;
DUMP B;
STORE TOP3 INTO 'OUT.TXT';
```

Optionális gyakorlat

Hadoop rendszer kipróbálása a para
klaszteren.

Hadoop on para.iit.bme

para: cluster on CIRCLE cloud

- **para (4 vCPU):**
 - Name Node,
 - Resource Manager,
 - Node Manager,
 - Data Node
- **cn01 (4 vCPU):**
 - Node Manager,
 - Data Node
- **cn02 (4 vCPU):**
 - Node Manager,
 - Data Node
- **cn03 (4 vCPU):**
 - Node Manager,
 - Data Node
- **cn04 (4 vCPU):**
 - Node Manager,
 - Data Node

Belépés, parncsok

ssh neptun@para.iit.bme.hu

module load hadoop # állítja a környezeti v.

hdfs dfs -

prefix után írt UNIX alapparancsok. Többé-
kevésbé hasonló paraméterekkel. pl.

hdfs dfs -ls /

Segédlet a Unix/Linux parancssori használatához:

https://infocpp.iit.bme.hu/unix_ea

https://infocpp.iit.bme.hu/sites/default/files/UXMLAB4_0.pdf

HDFS parancsok (példák)

```
hdfs dfs -mkdir /NEPTUN_KOD
```

```
hdfs dfs -ls /NEPTUN_KOD
```

```
hdfs dfs -cat /szebi/lorem.txt
```

```
hdfs dfs -get /szebi/lorem.txt
```

```
hdfs dfs -put lorem.txt /NEPTUN_KOD
```

```
hdfs dfs -ls /NEPTUN_KOD
```

```
hdfs dfs -rm /NEPTUN_KOD/lorem.txt
```

Monitorozó rendszer elérése

Szöveges böngészővel csak korlátozott funkcionálitással érhető el:

- HDFS státusz lekérdezése:
 - w3m <http://para:50070>
- YARN státusz lekérdezése:
 - w3m <http://para:8088>

A teljes funkcinonalitáshoz SSH tunnel kialakítása kell.

SSH tunnel kialakítás lépései

1. Kapcsolat létrehozása

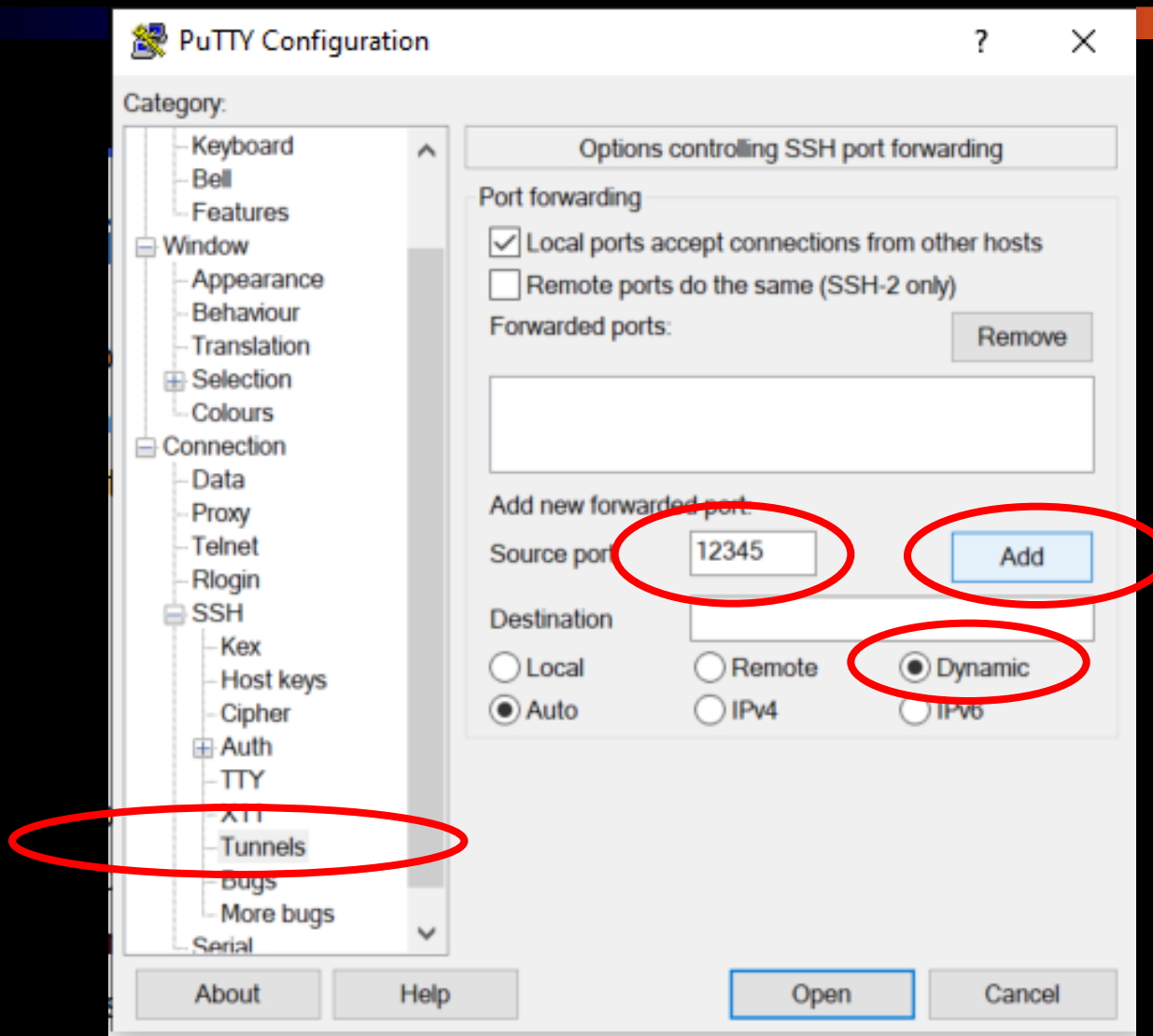
- putty-val

- ssh klienssel:

```
ssh -D 12345 NEPTUN@para.iit.bme.hu
```

2. böngésző beállítása

Putty beállítása



Firefox: Settings → Network Settings

The screenshot shows the 'Network Settings' window in Firefox. Several options are highlighted with red circles:

- ☒ **Manual proxy configuration**
- HTTP Proxy: [] Port: [0]
- ☐ Also use this proxy for HTTPS
- HTTPS Proxy: [] Port: [0]
- SOCKS Host: [127.0.0.1] Port: [12345]
- ☐ SOCKS v4 ☒ **SOCKS v5**
- ☐ Automatic proxy configuration URL
- [] Reload
- No proxy for: []
- Example: .mozilla.org, .net.nz, 192.168.1.0/24
- Connections to localhost, 127.0.0.1/8, and ::1 are never proxied.
- ☐ Do not prompt for authentication if password is saved
- ☒ **Proxy DNS when using SOCKS v5**

MapReduce példa futtatása

```
cp -r ~szebi/hadoop/wc .
```

```
cd wc
```

```
./compile_wc.sh
```

```
./run_wc.sh /szebi/lorem.txt /NEPTUN_KOD/out_1
```

```
hdfs dfs -ls /NEPTUN_KOD/out_1
```

```
hdfs dfs -cat /NEPTUN_KOD/out_1/part-r-00000
```

Segédanyagok

- **~szebi/hadoop**
 - Makefile – makefile a fordításhoz
 - WordCount.java – egyszerű wordcount
 - WordCount2.java – bővített változat. Kezeli az opciókat, kis-nagybetűt, skip fájlt.
- **/szebi (HDFS)**
 - HTTP-logs – apache logok
 - HTTP-logs2 – még több log
 - dict – szótárak
 - fiction – angol szövegek
- **<http://hadoop.apache.org/docs/current/api/org/apache/hadoop/mapred/package-summary.html>**

Pig példa futtatása

Kis adatokat, tesztet local módban érdemes:

```
module load hadoop
```

```
pig -x local ~szebi/hadoop/pig/sales.pig
```

Ha zavaró a sok kiírás

```
pig -4 ~szebi/hadoop/pig/nolog.conf -x local \  
~szebi/hadoop/pig/sales.pig
```

-x local nélkül MapReduce kódot generál. Ehhez a `sales_mp.pig` fájl kell!

Módosítási javaslat:

TT kiírása yyyy-MM-dd formában legyen!

TT kiírása totál szerint rendezett legyen!

Hive példa futtatása

Mindenki saját DB_**NEPTUNKÓD** nevű adatbázisban dolgozzon! Ehhez a hive parancsfájlban az összes @USER@ mintát le kell cserélni a saját kódra:

```
module load hadoop
```

```
cd; mkdir hive; cd hive
```

```
schematool -dbType derby -initSchema
```

```
sed -e 's/@USER@/NEPTUN/' \
```

```
~szebi/hadoop/hive/accesslog.hive > accesslog.hive
```

Hive parancsok futtatása:

```
hive -f accesslog.hive    # -S elnyomja a kiírásokat
```

Számítási és tárolási felhők (11.hét)

Felhők HPC és HTC környezetben

Szeberényi Imre, Ludmány Balázs

BME IIT

<szebi@iit.bme.hu>



MŰEGYETEM 1782

HPC Történelmi áttekintés

- Neumann korában felmerült az ötlet (Daniel Slotnick), de a csöves technológia nem tette ezt lehetővé.

Első szupergép:

- 1967-ben ILLIAC IV (256 proc, 200MFlop)
- Thinking Machines CM-1, CM-2 (1980)
- Cray-1

Párhuzamos feldolgozás?



Párhuzamosság = Egyidejűség

Azonos időben több dolgot végezni/számítani/csinálni

A korai számítógépek óta alkalmazzuk

I/O csatornák, DMA, periféria vezérlők, több ALU,
több CPUs, több mag, ...

Mire kell a nagy teljesítmény?

- Modellelés
 - időjárás, környezet, gyógyszerek, betegségek
 - előrejelzések (földrengés, tűzhányók, globális felmelegedés)
 - mechanika
- Szimulációk
 - tervezés, gyártás
 - bioinformatika, gyógyszerkutatás
 - részecskefizika

Fénysebességből adódó korlátok

A fénysebesség kb. 30 cm/ns.

Az elektromos jelek a fénysebesség 40-70%-ával terjednek.
Legyen ez most 15 cm/ns.

Ha egy jelnek 1.5 cm távolságot kell megtennie egy végrehajtási ciklus során aminek a hossza legyen 0.1 ns, akkor ebből max. 10 GIPS jön ki.

Ez a határ a miniatürizálással kitolható, de mindenképpen létező határ.

Teljesítmény növelés eszközei

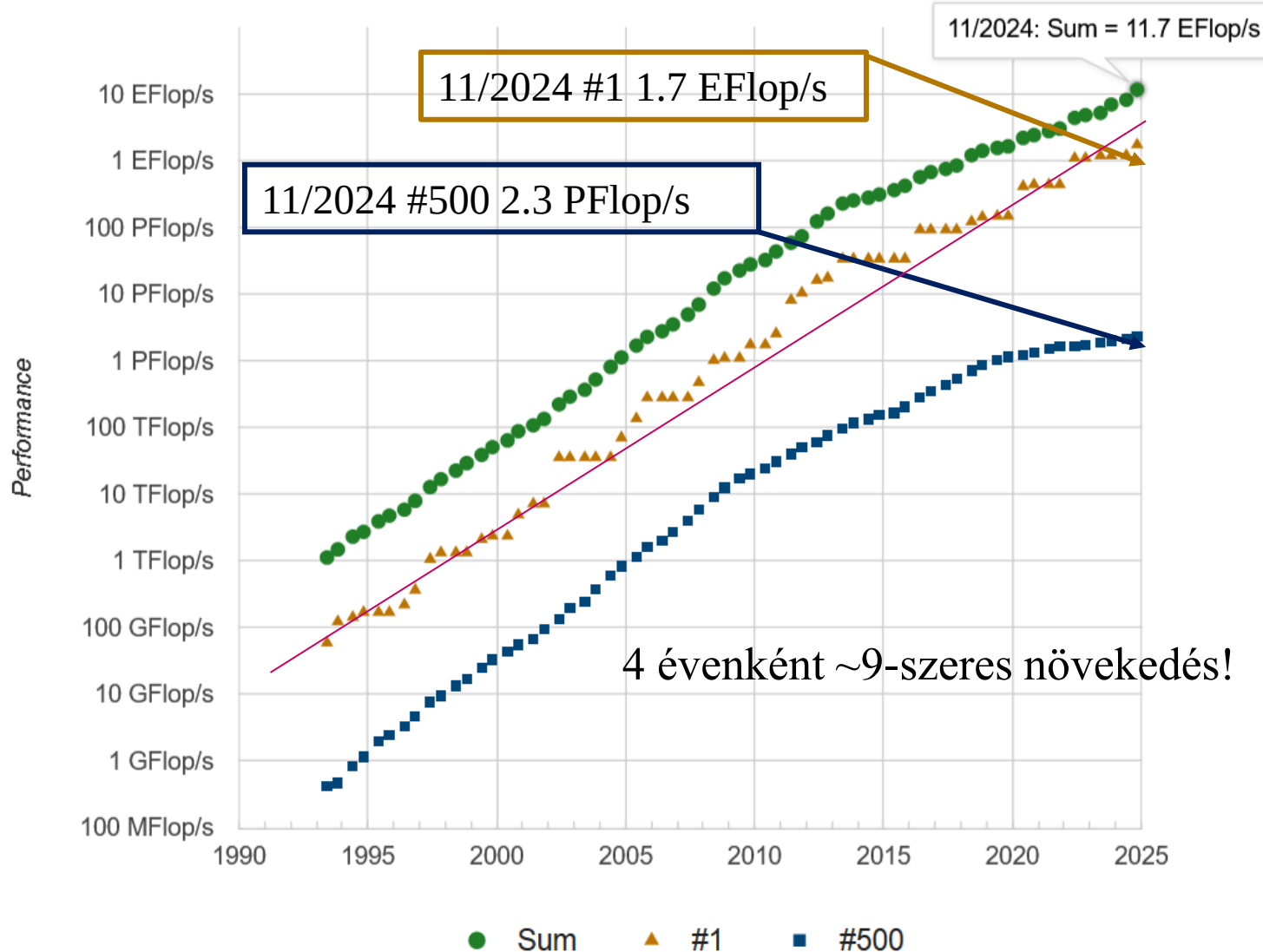
- Órajel növelése
 - fizikai korlátok (3.7GHz, nitrogén hűtéssel 9GHz)
 - energiaigény növekszik (köbös összefüggés)
- Párhuzamosítás
 - overhead
 - hibakeresés problémái

TOP 500 2024 june

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,206.00	1,714.81	22,786
2	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
3	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
4	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
5	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	531.51	7,107
6	Alps - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Swiss National Supercomputing Centre (CSCS) Switzerland	1,305,600	270.00	353.75	5,194

Only the
5th and
the 6th is
European

Performance Development



Fejlődés háttere

- Félvezetőgyártásban nagyobb elemsűrűség
 - 1971: 10 μ m, 1984: 1 μ m, 2001: 130nm, 2020: 5nm, 2022: 3nm, 2024 2nm
 - https://en.wikipedia.org/wiki/22_nm_process
 - <https://youtu.be/YIkMaQJSyP8>
- Hálózati technológiák fejlődése
- Tárolók fejlődése
- Memóriák fejlődése
- Sokmagos processzorok

Teljesítménymérés

Sebességnövekedés (Speed Up):

$$S_n = T_s / T_n$$

ahol: S_n N processzorral elért sebességnövekedés

T_s futási idő soros végrehajtás esetén

T_n futási idő N processzor esetén

Hatékonyság (Efficiency):

$$E_n = S_n / N$$

ahol: E_n N processzorral elért hatékonyság

S_n N processzorral elért sebességnövekedés

N processzorok száma

Redundancia (Redundancy):

$$r = C_p / C_s$$

ahol: r párhuzamos program redundanciája

C_p párhuzamos program műveleteinek száma

C_s soros program műveleteinek száma

Speed Up határa

Amdahl féle felső határ:

$$S_a = 1/(s+(1-s)/N)$$

ahol: S_a N processzorral elértetô sebességnövekedés felsô határa

s a feladat nem párhuzamosítható része

N processzorok száma

Az $(1-s)/N$ tagot elhagyva:

$$S_a < 1/s$$

összefüggést kapjuk, ami egy felső korlátot ad. Ez azt jelenti, hogy pl. 10% nem párhuzamosítható rész mellett $1/0.1 = 10$ adódik a speed up felső korlátjaként.

Párhuzamos gépek osztályai

- Vektor processzoros gépek
 - vektor jellegű adatokra optimalizált műveletek
 - egy operációs rendszer
- Szimmetrikus multiprocesszoros (SMP)
 - sok azonos processzor közös memóriával
 - egy operációs rendszerrel
 - NUMA, ccNUMA
- Masszívan párhuzamos (MPP)
 - sok processzor gyors belső hálózattal
 - elosztott memória
 - sok példányban fut az operációs rendszer
- Klaszter
 - sok gép gyors hálózattal összekötve
 - elosztott memória
 - sok példányban esetleg heterogén operációs rendszer

Jellemző architektúra



- 1993
 - SMP 49% (45%)
 - MPP 24% (36%)
 - Single procs. 20% (13%)
 - Vector 7% (6%)
- 2003
 - MPP 43% (51%)
 - Cluster 30% (35%)
 - Constellations 27% (14%) (SMP cluster)

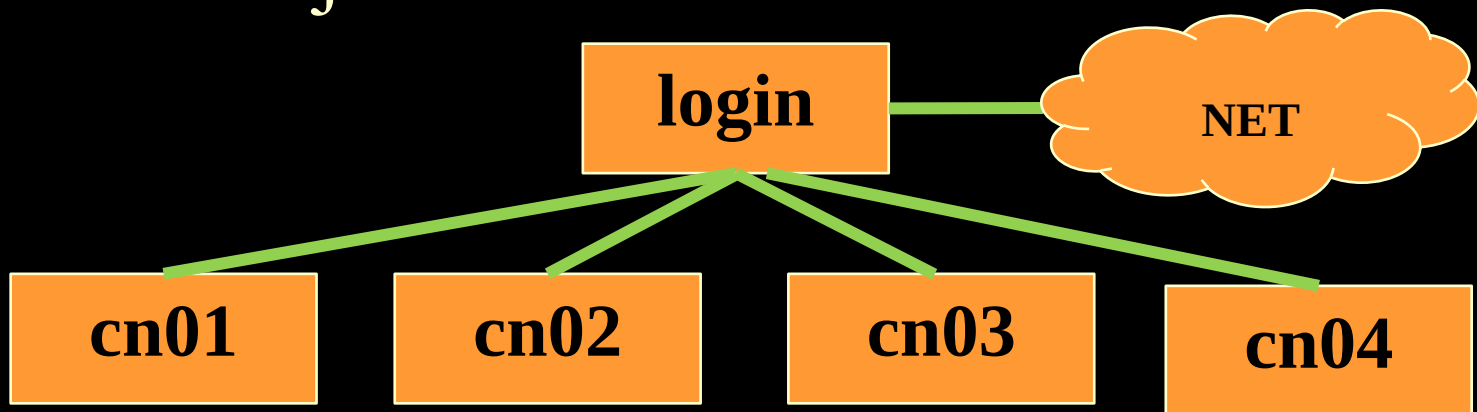
Jellemző architektúra #2



- 1913
 - Cluster 83% (60%)
 - MPP 17% (40%)
- 2023
 - Cluster 89% (54%)
 - MPP 11% (46%)
- 2024
 - Cluster 88% (51%)
 - MPP 12% (49%)

HPC klaszterek

- Egy vagy több login node
- Számos compute node
- Különböző partíciók
- Közös fájlrendszer



Hogy kerül a csizma az asztalra?

- Felhő környezet és a virtualizáció rugalmassága számos előnyt kínál az overhead-del szemben.
- Két irány:
 - Szupergépen belül container megoldások
 - Felhő + GPGPU → teljesítőképes virtuális klaszterek

Konténerek

Docker önmagában nem használható többfelhasználós környezetben.

- Apptainer/Singularity (<https://apptainer.org>)
- Shifter (<https://github.com/NERSC/shifter>)
- Charliecloud (<https://github.com/hpc/charliecloud>)
- Sarus (<https://github.com/eth-cscs/sarus>)
- ...

HPC felhőszolgáltatások

- AWS ParallelCluster
- Azure CycleCloud
- Google Cloud HPC Toolkit
- Oracle Cloud HPC
- HPE GreenLake
- Sabalcore HPC Cloud
- IBM Cloud HPC
- Alibaba Cloud HPC
- CoreWeave HPC Cloud

AI inference szolgáltatások

- OpenAI
- Anthropic
- Google Gemini
- AWS Bedrock
- Azure OpenAI Service
- Cohere
- Hugging Face Inference API
- CGG – (Compagnie Générale de Géophysique, 1931) → Viridien (HPC & AI cloud)

A modern AI-inference szolgáltatók nem HPC platformok, de sok esetben ugyanazokra a GPU-kapacitásokra épülnek.

HPC klaszterek problémái

- Változtatható környezet igénye:
 - környezeti változók, verziók kezelése
 - könyvtárak, alkalmazások telepítése
- Erőforrások allokálása és ütemezése
 - csak az adott felhasználó
 - véges erőforrások
- Közös fájlrendszerek
 - védelem

module

- Kissé elfeledett eszköz a környezeti változók gyors és karbantartható cseréjére, alternatív programváltozatok kiválasztására.

Példa:

```
module load mpi
printenv | grep MPI_LIB
MPI_LIB=/usr/local/openmpi/lib
module unload mpi
module load mvapich2
printenv | grep MPI_LIB
MPI_LIB=/usr/local/mvapich2/lib
```

module példa 2

module load mpi

module load cuda

module load dot

module list

Currently Loaded Modulefiles:

1) mpi 2) cuda 3) dot

A dot beteszi a PATH-ba az aktuális könyvtárat

Erőforrás elosztás igen fontos

- Erőforrás allokáció
- Erőforrás ütemezés
- Elosztott
- Lokális
- Sor alapú
- Illeszkedés alapú

Párhuzamos környezet

- Nem triviális az ütemezés
 - JOB A: 10 processzort igényel
 - JOB B: 4 processzort igényel
 - JOB C: 5 processzort igényel
 - 300 processzor foglalt 9 szabad
 - Ütemezés: ?????

Ütemezők



- Condor (Uni. of Wisconsin)
- DQS (Florida State Uni)
- LoadLeveler (IBM)
- Maui, Moab (Cluster Resources)
- LSF (Platform)
- PBS, OpenPBS (Alatair)
- Sun Grid Engine (SUN)
- Torque (Cluster Resources)
- SLURM (Simple Linux Utility for Resource Man)

SLURM

- Simple Linux Utility for Resource Management
- Erőforrások partíciókba vannak osztva
- Fontosabb parancsok:
 - sinfo – node és partíció infó
 - srun – párhuzamos job futtatása
 - sbatch – batch script futtatása
 - squeue – queue lekérdezése
 - salloc – erőforrás foglalás és interaktív futtatás
 - scancel – job törlése a sorból

SLURM példák /1

- `srun -n 2 /bin/hostname`
 - 2 CPU-t foglal és mindegyiken elindítja a `hostname` parancsot.
- `srun -N 2 /bin/hostname`
 - 2 node-ot foglal és mindegyiken elindítja a `hostname` parancsot.
- `salloc -n 4 /bin/bash`
 - 4 CPU-t foglal és elindítja az egyiken a `bash`-t interaktívan. Ebben pl. a `printenv | grep SLURM` parancs kiírja a beállított környezeti változókat.

SLURM példák /2

- `sbatch -n 3 lajoska.sh`
 - A várakozási sorban keletkezik egy olyan job, ami 3 CPU-t igényel és a parancs visszatér.
 - Amennyiben az erőforrások rendelkezésre állnak, lefoglalja azokat és az egyiken elindítja a `lajoska.sh`-t. A foglalt erőforrások neveit környezeti változóiban adja át.

Példa `lajoska.sh`:

```
#!/bin/bash
```

```
printenv | grep SLURM
```

```
hostname
```

Fontosabb kapcsolók

- n number of tasks to run
- ntasks-per-node=n number of tasks per node
- N number of nodes (N = min[-max])
- o location of stdout
- p partition requested
- prolog=program run "program" before job step
- i location of stdin
- t time limit
- mincpus=n minimum number of logical CPUs

Kapcsolók beépítése a szkriptbe

A kapcsolók egyszerű szintaxissal beépíthetők.

Legyen pl. a host.sh tartalma:

```
#!/bin/bash
#SBATCH -n 3
#SBATCH -o output_file
#SBATCH -D working_dir
#SBATCH --mail-type=end
#SBATCH --mail-user=xyz@xyz.de
#SBATCH -t 01:00:00
srun /bin/hostname
```

Az `sbatch host.sh srun -n 3` –ként fog futni, de

Az `sbatch -n 8 host.sh srun -n 8` –ként fog futni!

SPMD program futtatása

File: run.sh

```
#!/bin/bash
```

```
mpirun $@
```

Parancs:

```
sbatch -n 4 run.sh ./mpihello
```

File: runhello.sh

```
#!/bin/bash
```

```
#SBATCH -o hello.txt
```

```
#SBATCH -n 4
```

```
mpirun ./mpihello
```

Parancs:

```
sbatch runhello.sh
```


MPMD modell futtatása

File: multi.sh

```
#!/bin/bash
```

```
#SBATCH -N 2
```

```
#SBATCH -n 8
```

```
#SBATCH -o multi.out
```

```
srun --multi-prog ./multi.conf
```

File: multi.conf

```
0 ./master.sh
```

```
## slaves
```

```
* ./slave.sh
```

Parancs:

```
sbatch multi.sh
```

Fontosabb környezeti változók:

SLURM_PROCID

SLURM_NODEID

SLURM_LOCALID

Számítási és tárolási felhők alapjai

Infrastructure-as-Code

Ludmány Balázs, Szeberényi Imre

2025. 11. 25.

Infrastruktúra felügyelet és konfigurációmenedzsment

- ▶ „Hagyományos” megoldás: SSH-n keresztül szkripteket futtatunk a VM-eken
 - ▶ rosszul skálázódik
 - ▶ átmásolhatók-e egy szkriptet egy másik projektbe?
 - ▶ jelentős az emberi hiba lehetősége
 - ▶ nehéz reprodukálni
 - ▶ inkonzisztencia pl. a teszt és az éles rendszer közt
- ▶ *Infrastructure-as-Code (IaC)*: írjuk le kóddal az infrastruktúrát is
 - ▶ verziókövetés
 - ▶ code review
 - ▶ jól ismert dokumentációs megoldások
 - ▶ ugyanabból a konfigurációs fájlból ugyanazt a környezetet kapjuk

laC alapelvek

Deklaratív konfiguráció nem a rendszeren végzendő változtatásokat írjuk le, hanem a *kívánt állapotot*

Automatizáció fejlesztőként kizárólag a kívánt állapotot módosítjuk a deklaratív konfigurációs fájlokban, az infrastruktúrát az laC eszköz módosítja

Folyamatos állapotmenedzsment az laC eszköz folyamatosan figyeli az infrastruktúra állapotát, és beavatkozik, ha az eltér a kívánt állapottól

Idempotens működés ugyanazt az állapotot kapjuk, ha egy módosítást többször alkalmazunk

Reprodukálhatóság és konzisztencia ugyanabból a konfigurációs fájlból ugyanazt a környezetet kapjuk, amik később is konzisztensek maradnak

Modularitás és hordozhatóság az általánosabb beállításokat modulokba szervezzük és több projektben felhasználjuk

Működési elv

- ▶ *Erőforrás gráf*: épít egy irányított körmentes gráfot (DAG) a rendszer aktuális állapotáról és a kívánt állapotról a konfigurációs fájlok alapján, majd a különbségük alapján beavatkozik.
- ▶ A beavatkozás lehet
 - push elvű* a konfigurációs szerver módosítja az erőforrásokat
 - pull elvű* az erőforrások lekérlik a saját beállításait
- ▶ *Immutabilitás*: bizonyos esetekben egyszerűbb a meglévő erőforrások módosítása helyett új konfigurációval új példányokat indítani, majd a régieket törölni (pl. VM-ek)

IaC eszközök

Infrastruktúra felügyelet

VM, Kubernetes klaszter stb. létrehozása, a felhőszolgáltatásokhoz való hozzáférési jogosultságok kezelése. . .

- ▶ Terraform
- ▶ OpenTofu
- ▶ AWS CloudFormation

Konfigurációmenedzsment

Szoftverek telepítése és konfigurálása az infrastruktúrán

- ▶ Ansible
- ▶ Puppet
- ▶ Chef
- ▶ SaltStack

Terraform

- ▶ 2014, HashiCorp
- ▶ Push elvű infrastruktúra felügyelet
- ▶ AWS, Azure, GCP, OpenStack stb.
- ▶ Goban íródott
- ▶ HashiCorp Configuration Language (HCL)
- ▶ Eredetileg Mozilla Public License (MPL): nyílt forráskódú, szabad szoftver
- ▶ 2023-tól Business Source License (BUSL): source-available, korlátozott kereskedelmi használat
- ▶ *OpenTofu*: az utolsó MPL változat forkja a Linux Foundation felügyelete alatt

Terraform terminológia

leírás	példa
szolgáltatókat (<i>provider</i>) plugineken keresztül ér el	AWS, Azure, OpenStack
a szolgáltatókat a <i>registry-ből</i> tölti le	Terraform Registry vagy privát
a Terraform a szolgáltatónál elérhető <i>erőforrásokat</i> (<i>resource</i>) felügyeli	IAM felhasználó, EC2 VM, EKS klaszter
a saját .tf kiterjesztésű konfigurációs fájljaink alkotnak egy <i>modult</i>	main.tf
a <i>változók</i> (<i>variable</i>) egy modul argumentumai	
használhatunk csak olvasható <i>adatokat</i> (<i>data</i>)	már futó VM azonosítója
generálhatunk <i>kimenetet</i> (<i>output</i>)	a modulban létrehozott VM azonosítója

Terraform munkamenet

- ▶ A Terraform CLI az aktuális könyvtárban minden .tf fájlt feldolgoz.
- ▶ Mélyebbre viszont nem megy, az alkönyvtárak tartalmát külön moduloknak tekinti.
- ▶ Fő parancsok:
 - `terraform init` inicializálja a munkakönyvtárat (pl. provider plugin letöltése)
 - `terraform plan` tervet készít, hogy a rendszer hogy kerülhet a konfigurációban megadott kívánt állapotba
 - `terraform apply` beavatkozik, hogy a rendszer a kívánt állapotba kerüljön
 - `terraform output` a végrehajtás során generált kimenet elérése
- ▶ Mindig érdemes először átnézni a `plan` kimenetét, és csak utána módosítani az `apply`-jal.

Terraform példa – AWS IAM felhasználó létrehozása I

variables.tf

```
# régiót tároló változó
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "eu-north-1"
}
```

main.tf

```
# AWS provider a Terraform Registryből
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# régió beállítása változóból
provider "aws" {
  region = var.aws_region
}
```

Terraform példa – AWS IAM felhasználó létrehozása II

main.tf – folytatás

```
# a felügyelt szabályzat adatainak lekérése
data "aws_iam_policy" "ec2_full_access" {
  name = "AmazonEC2FullAccess"
}

# felhasználó létrehozása
resource "aws_iam_user" "developer" {
  name = "developer"
}

# szabályzat hozzárendelése a felhasználóhoz
resource "aws_iam_user_policy_attachment" "ec2_full_access" {
  user          = aws_iam_user.developer.name
  policy_arn    = data.aws_iam_policy.ec2_full_access.arn
}
```

Terraform példa – AWS IAM felhasználó létrehozása III

`outputs.tf`

```
# kimenet
output "iam_user_name" {
  description = "The name of the created IAM user"
  value       = aws_iam_user.developer.name
}
output "iam_user_arn" {
  description = "The ARN of the created IAM user"
  value       = aws_iam_user.developer.arn
}
```

Terraform példa – AWS IAM felhasználó létrehozása IV

Telepítés

```
$ terraform init
$ terraform plan -var="aws_region=us-east-1" -out=iam_tfplan
$ terraform apply iam_tfplan
```

Terraform példa – EC2 példány létrehozása I

Az első dia ugyanaz, mint az előző példában

variables.tf

```
# régiót tároló változó
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "eu-north-1"
}
```

main.tf

```
# AWS provider a Terraform Registryből
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# régió beállítása változóból
provider "aws" {
  region = var.aws_region
}
```

Terraform példa – EC2 példány létrehozása II

main.tf – folytatás

```
# Amazon Linux 2023 AMI megkeresése
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name     = "name"
    values   = ["al2023-ami-*-kernel-*-x86_64"]
  }
}
```

Terraform példa – EC2 példány létrehozása III

main.tf – folytatás

```
# SSH hozzáférés engedélyezése a 22-es porton
# FIGYELEM: éles környezetben nem biztonságos
resource "aws_security_group" "allow_ssh" {
  name          = "allow_ssh"
  description   = "Allow SSH inbound"
  ingress {
    from_port    = 22
    to_port      = 22
    protocol     = "tcp"
    cidr_blocks  = ["0.0.0.0/0"]
  }
  egress {
    from_port    = 0
    to_port      = 0
    protocol     = "-1"
    cidr_blocks  = ["0.0.0.0/0"]
  }
}
```


Terraform példa – EC2 példány létrehozása IV

main.tf – folytatás

```
# t3.micro példány létrehozása
resource "aws_instance" "amazon_linux_2023" {
  ami           = data.aws_ami.al2023.id
  instance_type = "t3.micro"
  security_groups = [aws_security_group.allow_ssh.name]
}
```

Terraform példa – EC2 példány létrehozása V

outputs.tf

```
# kimenet
output "instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.amazon_linux_2023.id
}

# érzékeny kimenet
output "public_ip" {
  description = "EC2 public IP"
  value       = aws_instance.amazon_linux_2023.public_ip
  sensitive   = true
}
```

Terraform példa – EC2 példány létrehozása VI

Telepítés

```
$ terraform init
$ terraform plan
$ terraform apply
# az érzékeny adatokat nem írja a kimenetre
# explicit le kell kérni
$ terraform output -raw public_ip
```

Ansible

- ▶ 2013, Michael DeHaan
- ▶ Push elvű konfigurációmenedzsment
- ▶ Különböző Linux disztribúciók, BSD variánsok és Windows támogatása
- ▶ Pythonban íródott
- ▶ YAML konfigurációs fájlok
- ▶ GNU General Public License (GPL): nyílt forráskódú, szabad szoftver
- ▶ 2015-ben az Ansible, Inc.-et felvásárolta a RedHat

Ansible terminológia

- ▶ *Vezérlő (control)* és *felügyelt (managed)* node-ok.
- ▶ A felügyelt node-ok IP címeit vagy hosztneveit a vezérlő node-on elhelyezett *inventory* konfigurációs fájl tartalmazza.
- ▶ A *play* egy adott felügyelt node-on elvégzendő *feladatokat (task)* tartalmazza.
- ▶ A play-eket tartalmazó konfigurációs fájl a *playbook*.
- ▶ A *modul* egy bizonyos feladatot elvégző szkript/bináris.
- ▶ *Agentless* architektúra: nem fut folyamatosan egy folyamat a felügyelt node-on. Minden alkalommal be-SSH-zik, átmásolja, futtatja majd törli a szükséges modulokat.

Ansible példa – webservertelepítése I

- ▶ A vezérlő node-on dolgozunk
- ▶ Az inventory-ban megadjuk a felügyelt node (node-ok) IP címét és a bejelentkezéshez szükséges SSH kulcsot:

`inventory/webservers.ini`

```
[webservers]
web1 ansible_host=10.0.1.100 ansible_user=ec2-user \
ansible_ssh_private_key_file=~/.ssh/ec2-key.pem
```

Ansible példa – webservertelepítése II

apache-install.yaml

```
- name: Install Apache on Amazon Linux 2023
  # webservers.ini-ből
  hosts: webservers
  # root-ként fut, hogy használhassa a csomagkezelőt
  become: yes
```

tasks:

```
- name: Install Apache
  yum:
    name: httpd
    state: present
```

Ansible példa – webservertelepítése III

apache-install.yaml – folytatás

- name: Start Apache
systemd:
 name: httpd
 state: started
 enabled: yes
- name: Create index.html
copy:
 content: "<h1>Apache is running!</h1>"
 dest: /var/www/html/index.html

Ansible példa – webserverver telepítése IV

Telepítés

```
$ ansible-playbook -i inventory/webservers.ini apache-install.yaml
```

Terraform és Ansible integrációja

- ▶ A két eszközt integráltan is lehet használni:
 - Terraform létrehozza az infrastruktúrát (VM-ek, hálózatok stb.)
 - Ansible telepíti és konfigurálja az infrastruktúrán futó szoftvert
- ▶ Egy Terraform modul akár meg is hívhat egy Ansible playbookot.